

Impact of Prompt Formatting on Instruction Adherence in Large Language Models

Batuhan Yılmaz
Dept. of Computer Engineering
Akdeniz University
Antalya, Turkey
20220808068

Emre Kemal Aksel
Dept. of Computer Engineering
Akdeniz University
Antalya, Turkey
20220808065

Abstract—This report investigates the impact of structural prompt formatting (Plain Text, Markdown, XML, JSON, and YAML) on the instruction adherence capabilities of Large Language Models (LLMs). Instruction adherence was evaluated using two primary benchmarks: IFEval and IFBench. Due to excessive computational overhead, LLM-as-a-judge frameworks such as WildBench were bypassed in favor of deterministic, programmatically verifiable evaluation. We detail our extensive model selection journey, migrating from Qwen 3.5 9B and Gemini 2.5 Flash to our final model, DeepSeek v3.1, while addressing inference bottlenecks such as infinite reasoning loops, degenerative repetition, and context window overflows. Our findings indicate that for shorter tasks with strict verifiable rules (IFEval), XML formatting yields the highest constraint adherence (89.28% strict accuracy). Conversely, for longer and more complex out-of-distribution instructions (IFBench), the distinction between structural formats diminishes, with JSON marginally outperforming the others (50.67% strict accuracy). These results highlight the complex correlation between task context length, constraint strictness, and optimal prompt structuring.

Index Terms—Large Language Models, Prompt Engineering, Instruction Following, IFEval, IFBench

I. PROBLEM DEFINITION AND MOTIVATION

As Large Language Models (LLMs) are increasingly integrated into agentic workflows and complex applications, developers rely heavily on structural prompt syntax (e.g., XML, JSON, Markdown) to guide the model’s behavior. Despite this reliance, there is limited empirical consensus on which formatting style objectively yields the highest level of constraint adherence.

This project addresses this gap by systematically evaluating how prompt formatting impacts an LLM’s ability to follow complex, multi-step instructions. We built our evaluation pipeline upon the foundational Instruction Following Evaluation (IFEval) benchmark [1], which tests strict verifiable rules (e.g., specific word counts or forbidden letters), and IFBench [2], which introduces harder, out-of-distribution (OOD) constraints. We synthesized five exact format permutations (Plain Text, Markdown, XML, JSON, YAML) for each baseline prompt to strictly isolate the syntax variable. This report details both our quantitative findings and the complex computational and architectural bottlenecks we had to overcome to generate responses with modern reasoning models.

Project Repository: <https://github.com/BatY0/LLMTermProject>

II. DATASET AND DATA SOURCE

Evaluating LLM instruction adherence requires reliable, scalable benchmarks. Our primary data sources were IFEval [1] and IFBench [2]. IFEval provides a highly deterministic framework utilizing Python scripts to programmatically verify whether an LLM followed constraints, eliminating human subjectivity. IFBench extends this by introducing longer, more challenging out-of-domain constraints to test a model’s true generalization capabilities.

To properly isolate the formatting variable, every prompt in these selected benchmarks needed to be meticulously converted into five distinct variations: Plain, Markdown, XML, JSON, and YAML. Because automated scripts proved insufficient for cleanly separating core tasks from explicit constraints in complex prompts, we altered the data sequentially. We developed a custom HTML-based interface to manually iterate through the dataset item-by-item. This interface allowed us to restructure the raw prompts into a task/rule-oriented representation, clearly demarcating the task content from the rule content, and subsequently outputting a newly structured JSON dataset containing the five formatting variants.

III. LLM METHODOLOGY

A. Model Selection and Inference Challenges

The process of generating text outputs across all formatting variants involved a highly iterative model selection journey to overcome computational bottlenecks:

- **Attempt 1: Qwen 3.5 9B (Google Colab).** We initially attempted to run inference locally using Qwen 3.5 9B on a free tier Google Colab instance. Despite optimizations, the execution speed indicated it would take approximately 51 hours to process a single dataset, rendering it incompatible with Colab’s 4-hour session limit.
- **Attempt 2: Qwen 3.5 9B (OpenRouter API).** Transitioning to the OpenRouter API bypassed local hardware limits but introduced severe bottlenecks associated with the model’s “thinking” (Chain-of-Thought) process. Extended reasoning frequently led to context overflow, causing the model to abruptly restart its generation from zero and trapping us in an endless loop of retries.

- **Attempt 3: Gemini 2.5 Flash.** We switched to Gemini 2.5 Flash to increase inference speed. While throughput vastly improved, the model suffered from qualitative logic issues. It exhibited degenerative repetition, caught itself in loops, and ultimately overflowed its own context length limit.
- **Final Model: DeepSeek v3.1.** To balance reasoning capability with execution stability, we finalized our inference pipeline using DeepSeek v3.1. To mitigate the risk of the model entering an infinite reasoning loop, we implemented a strict token limit of 4096. This successfully resolved the repetition issues and stabilized our dataset generation.

To guarantee deterministic behavior and fairly compare the structural formatting variants, our prompting strategy enforced a generation temperature of 0 across all inference runs. This ensured that the model produced the exact same logical pathways for identical prompts, isolating the prompt structure as the sole independent variable.

B. Evaluation Pipeline

Once the model outputs were aggregated via OpenRouter, we formatted the responses into benchmark-compatible JSONL files. We modified the standard IFEval and IFBench grading scripts so they could automatically evaluate and aggregate both strict and loose accuracy scores across our five custom syntax groupings.

IV. EXPERIMENTS AND RESULTS

A. IFEval Results

IFEval features relatively short tasks bounded by highly strict formatting rules. As shown in Table I, the XML format achieved the highest performance by a notable margin, reaching an 89.28% Strict Accuracy score. In shorter contexts with rigid constraints, XML provides the clearest structural boundaries, allowing the LLM to effectively isolate rules from task content.

TABLE I
IFEVAL PROMPT-LEVEL ACCURACY

Rank	Format	Strict (%)	Loose (%)
1	XML	89.28	91.87
2	YAML	85.95	87.98
3	Markdown	85.76	88.90
4	JSON	85.21	88.53
5	Plain	85.02	88.72

B. IFBench Results

IFBench comprises longer tasks with fewer explicit formatting instructions but highly complex out-of-domain constraints. As observed in Table II, overall performance dropped drastically across all formats, reflecting the benchmark’s difficulty. Here, JSON emerged as the highest-performing format, achieving a 50.67% Strict Accuracy.

TABLE II
IFBENCH PROMPT-LEVEL ACCURACY

Rank	Format	Strict (%)	Loose (%)
1	JSON	50.67	52.67
2	Plain	48.67	51.33
3	Markdown	47.00	48.67
4	XML	46.67	49.33
4	YAML	46.67	49.33

C. Example Outputs and Error Analysis

To illustrate the formatting differences, consider an IFEval prompt requiring a response without using the letter ‘a’. In the Plain Text format, the constraint is appended as a simple sentence: “Do not use the letter a.” In the XML format, the constraint is wrapped as `<constraint>Do not use the letter a.</constraint>`. When evaluated, the XML wrapper successfully forced DeepSeek to attend strictly to the negative constraint. Conversely, the Plain Text variant occasionally caused the model to overlook the rule during its generative process, resulting in a failed constraint.

Our error analysis revealed two primary failure modes. First, in non-XML formats (particularly Markdown and Plain Text), the model occasionally merged the constraint instructions with the task persona, leading to errors where the model would state the rules aloud in its output rather than seamlessly following them. Second, in IFBench’s longer tasks, the model frequently failed complex multi-step formatting requirements because the prompt constraints were buried within dense paragraphs, making structural delimiters like XML or JSON less effective at drawing the model’s attention.

V. DISCUSSION AND LIMITATIONS

A critical observation from our results is the behavioral shift between the two benchmarks. In IFEval, the distinct syntax of XML drastically improved adherence. However, in IFBench, the performance gap between the best format (JSON) and the worst (XML/YAML) narrowed to just 4%. This suggests that in longer tasks, the structural wrapper plays a less significant role as the LLM is forced to manage larger cognitive loads over extended context windows.

Our study presents a few notable limitations. Firstly, due to computational and financial budget constraints, we excluded LLM-as-a-judge frameworks like WildBench [3]. Evaluating five prompt permutations per dataset item using a dual-agent architecture proved infeasible, limiting our scope to programmatically deterministic benchmarks. Secondly, the 4096-token limitation we enforced on DeepSeek v3.1, while strictly necessary to prevent repetition and endless reasoning loops, restricted the model’s internal Chain-of-Thought (CoT) capacity. This truncation likely impacted the model’s ability to plan its responses, contributing to the lower overall scores on the more complex IFBench dataset compared to IFEval.

VI. CONCLUSION

This project systematically evaluated how five distinct prompt formats impact LLM instruction adherence. Through extensive iterative testing, we concluded that structural prompt formatting heavily influences model performance on short, rule-strict tasks, with XML yielding the highest fidelity. However, as task length and complexity increase, the performance distinction between structural wrappers shrinks. Deploying modern reasoning models requires a careful balance; while expansive context windows allow for deep reasoning, capping token limits is often a necessary tradeoff to prevent inference bottlenecks. Ensuring a temperature of 0 provided a solid baseline, verifying that these observed format-driven differences were consistent and measurable rather than byproducts of stochastic generation.

REFERENCES

- [1] J. Zhou, T. Lu, S. Mishra, S. Brahma, S. Basu, Y. Luan, D. Zhou, and L. Hou, "Instruction-Following Evaluation for Large Language Models," *arXiv preprint arXiv:2311.07911*, 2023. <https://arxiv.org/abs/2311.07911>
- [2] AllenAI / V. Pyatkin et al., "Generalizing Verifiable Instruction Following" (IFBench), *arXiv preprint*, 2025. [Online]. Available: <https://arxiv.org/pdf/2512.09663>
- [3] B. Y. Lin, Y. Deng, K. Chandu, F. Brahman, A. Ravichander, V. Pyatkin, N. Dziri, R. Le Bras, and Y. Choi, "WildBench: Benchmarking LLMs with Challenging Tasks from Real Users in the Wild," *arXiv preprint arXiv:2406.04770*, 2024. <https://arxiv.org/abs/2406.04770>